

Highly Available Web Infrastructure on AWS

with Auto Scaling and CDN

AWS Cloud Infrastructure Project

Project by: Mohammad Maheer Shaik

Region: eu-north-1 (Stockholm)

Date Completed: May 2026

1. Project Overview

This project demonstrates the deployment of a production-grade, highly available and scalable web infrastructure on Amazon Web Services (AWS). The architecture spans multiple Availability Zones and integrates core AWS services including load balancing, auto scaling, a CDN, shared file storage, a managed relational database, and DNS configuration.

2. AWS Services Used

#	Service	Purpose
1	Amazon VPC	Custom isolated network with defined CIDR block
2	Amazon EC2	Virtual web server instances running NGINX
3	Amazon EFS	Shared NFS file system mounted across EC2 instances
4	Amazon RDS	Managed MySQL relational database
5	Amazon S3	Static asset and file storage
6	Amazon CloudFront	Global CDN for low-latency content delivery
7	Application Load Balancer	Distributes HTTP traffic across instances
8	Auto Scaling Group	Automatic instance provisioning and management
9	Amazon Route 53	DNS hosted zone configuration
10	AWS Certificate Manager	SSL/TLS certificate request and DNS validation configuration
11	DuckDNS	Free subdomain used for testing and access

3. Infrastructure Details

3.1 VPC

Field	Value
VPC ID	vpc-0686700b8c9b73fac
Name	Bproject-vpc
IPv4 CIDR	10.0.0.0/16
DNS Hostnames	Enabled
Region	eu-north-1 (Stockholm)

3.2 Subnets

Name	Subnet ID	CIDR	Availability Zone
Bproject-subnet-public1-eu-north-1a	subnet-0574b67d034ab1aa6	10.0.0.0/20	eu-north-1a

Name	Subnet ID	CIDR	Availability Zone
Bproject-subnet-public2-eu-north-1b	subnet-0a892625e6c41ac5a	10.0.16.0/20	eu-north-1b

3.3 EC2 Instances

Name	Instance ID	Type	AZ	Private IP
web-server-1	i-09666078d1d7c2fc7	t3.micro	eu-north-1a	10.0.3.90
web-server-2	i-0016d6b112fbf7311	t3.micro	eu-north-1b	10.0.24.66

3.4 Amazon EFS

Field	Value
File System ID	fs-027064200a0bed4cf
Name	Bproject-efs
Performance Mode	General Purpose
Throughput Mode	Elastic
Availability Zone	Regional (multi-AZ)
DNS Name	fs-027064200a0bed4cf.efs.eu-north-1.amazonaws.com
Automatic Backups	Enabled
Encrypted	Yes

3.5 Amazon RDS

Field	Value
DB Identifier	projectdb
Engine	MySQL Community
Instance Class	db.t3.micro
Region & AZ	eu-north-1a
Status	Available
Internet Access Gateway	Disabled (private access only)
Inbound Security Group	rds-sg — allows MySQL from EC2 SG only

3.6 Amazon S3

Field	Value
Bucket Name	bproject-s3bucket-mahi
Objects	static.html, reddy's.jpg

Field	Value
Region	eu-north-1
Access	Configured for CloudFront origin

3.7 Auto Scaling Group

Field	Value
ASG Name	Bproject-asg
Launch Template	Bproject-temp (lt-06aac56f0f565c3b0)
AMI	ami-0d1856c2239281718 (Bproject-ami)
Instance Type	t3.micro
Desired Capacity	2
Minimum Capacity	2
Maximum Capacity	4
Status	At desired capacity

3.8 CloudFront Distribution

Field	Value
Distribution ID	E1W50SJAZUBD5N
Domain Name	dhxsjd0yojfl7.cloudfront.net
Price Class	All Edge Locations (Best Performance)
HTTP Versions	HTTP/2, HTTP/1.1, HTTP/1.0
Default Root Object	index.html
Origin	Application Load Balancer (Bproject-alb)

3.9 Route 53 & DNS

Field	Value
Hosted Zone Name	mybproject.duckdns.org
Hosted Zone ID	Z029097633R8WPOFW1C2U
Type	Public Hosted Zone
CNAME Record	Points to CloudFront distribution
Name Servers	ns-1849.awsdns-39.co.uk, ns-801.awsdns-36.net, ns-1132.awsdns-13.org, ns-469.awsdns-58.com

4. Architecture Flow

- DNS records were configured for testing using DuckDNS and Route 53
- CloudFront serves cached content from the nearest edge location
- For dynamic requests, CloudFront forwards traffic to the Application Load Balancer
- ALB distributes requests across EC2 instances in eu-north-1a and eu-north-1b
- EC2 instances (web-server-1 and web-server-2) serve the web application via NGINX
- All EC2 instances read/write shared web content from EFS mounted at /var/www/html/
- Application data is stored in and retrieved from RDS MySQL (projectdb)
- Static assets (images, HTML) are stored in S3 and served directly via CloudFront
- Auto Scaling Group monitors instance health and maintains 2–4 instances at all times

5. Implementation Steps

Step 1 — VPC Configuration

- Created custom VPC Bproject-vpc with CIDR 10.0.0.0/16
- Created two public subnets across eu-north-1a (10.0.0.0/20) and eu-north-1b (10.0.16.0/20)
- Attached Internet Gateway Bproject-igw to the VPC
- Configured route table Bproject-rtb-public: 0.0.0.0/0 → IGW and 10.0.0.0/16 → local
- Enabled auto-assign public IPv4 on both public subnets

Step 2 — EC2 Web Servers

- Launched t3.micro Amazon Linux 2023 EC2 instances in each public subnet
- Installed NGINX: `sudo dnf install -y nginx`
- Created and deployed custom index.html web pages
- Configured NGINX to serve the application on port 80

Step 3 — EFS Shared Storage

- Created Regional EFS file system Bproject-efs (General Purpose, Elastic throughput)
- Configured mount targets in both eu-north-1a and eu-north-1b subnets
- Installed NFS utilities on EC2: `sudo dnf install -y amazon-efs-utils`
- Mounted EFS on /var/www/html/ using the NFS mount command
- Recreated index.html on the EFS mount so all instances serve identical content

Step 4 — RDS MySQL Database

- Launched MySQL RDS instance projectdb (db.t3.micro) in eu-north-1a
- Configured security group rds-sg to allow port 3306 inbound only from EC2 security group
- Installed mysql-client on EC2: `sudo dnf install -y mysql`
- Verified connectivity: `mysql -h <RDS-endpoint> -P 3306 -u admin -p`

Step 5 — S3 Static Storage

- Created S3 bucket bproject-s3bucket-mahi in eu-north-1
- Uploaded static website assets: static.html and reddy's.jpg
- Configured bucket policy and permissions for CloudFront origin access

Step 6 — Application Load Balancer

- Created ALB Bproject-alb spanning both public subnets
- Created target group Bproject-tg (HTTP:80, Instance type, HTTP1)
- Registered web-server-1 and web-server-2 as targets
- Verified both registered targets reported as Healthy

Step 7 — AMI and Auto Scaling

- Created custom AMI Bproject-ami (ami-0d1856c2239281718) from the configured EC2 instance
- Created Launch Template Bproject-temp using the custom AMI and security group
- Configured Auto Scaling Group Bproject-asg with min=2, desired=2, max=4
- Linked ASG to ALB Target Group for automatic health check integration
- Verified ASG maintained desired capacity and registered new instances with ALB

Step 8 — CloudFront CDN

- Created CloudFront distribution Bproject-cf with ALB as origin
- Configured CloudFront distribution with Application Load Balancer as origin
- Set index.html as the default root object
- Enabled all edge locations for best performance
- Verified CloudFront to ALB routing and web accessibility

Step 9 — DNS Configuration

- Registered subdomain mybproject.duckdns.org via DuckDNS
- Created Route 53 public hosted zone for mybproject.duckdns.org
- Created CNAME record pointing to CloudFront distribution endpoint
- Requested SSL certificate via AWS Certificate Manager and created DNS validation CNAME record
- Configured Route 53 hosted zone and DNS records for testing

Step 10 — Verification

- Verified CloudFront to ALB routing and web accessibility
- Verified CloudFront → ALB → EC2 request routing
- Confirmed EFS shared storage consistency across both EC2 instances
- Tested RDS MySQL database connectivity from EC2
- Verified Auto Scaling Group maintained healthy instance count

6. Challenges Faced

Challenge	Description
CloudFront Origin Configuration	Troubleshooting the correct origin path and protocol settings when connecting CloudFront to the ALB. Required testing to ensure requests were forwarded correctly without path errors.
DNS Validation Limitations with DuckDNS	DuckDNS does not support full nameserver delegation to Route 53, which prevented complete HTTPS custom domain validation via ACM. DNS records were configured for testing purposes only.

Challenge	Description
NGINX Default Root Object	The NGINX default server root had to be aligned with the EFS mount point (/var/www/html/) to ensure CloudFront and the ALB served the correct index.html file.
EFS Mount Consistency	After mounting EFS on both EC2 instances, the index.html file needed to be recreated on the shared mount point to ensure both instances served identical content.
Security Group Configuration for RDS	Ensuring the RDS security group only allowed inbound MySQL traffic from the EC2 security group (not open to the internet) required careful inbound rule configuration.

7. Key Features

Feature	Implementation
High Availability	Instances deployed across eu-north-1a and eu-north-1b
Load Balancing	ALB distributes traffic evenly across AZs
Auto Scaling	ASG scales between 2 and 4 instances based on demand
Shared Storage	EFS ensures all instances serve identical content
CDN Integration	CloudFront delivers content from global edge locations
CloudFront CDN	Configured for content delivery via global edge locations
Managed Database	RDS MySQL isolated behind EC2-only security group rules
Custom DNS	Route 53 + DuckDNS domain mapped to CloudFront